

Sorting Algorithm Analysis

The Sorting Problem

Before analyzing any algorithm, We must state the problem precisely.

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that:

$$a'_1 \leq a'_2 \leq a'_3 \leq \dots \leq a'_n$$

The numbers we wish to sort are also known as **keys**. In practice, each key usually comes attached to **satellite data** — other information associated with that **key**. For example, sorting a list of students by GPA: the GPA is the key, but each student has a name, ID, course history, etc. The sorting algorithm moves keys around, and satellite data travels with them.

The Computational Model

RAM (Random Access Machine) model:

- Instructions execute one at a time, sequentially
- Each basic operation (arithmetic, comparison, assignment, array access, control flow) takes *constant time*
- Memory access takes constant time regardless of location
- We do *not* model cache effects, pipelining, or parallelism

This is an abstraction, but a useful and standard one. It lets us count operations cleanly.

What We Count

We measure running time as a *function of input size n* . The running time is the *number of primitive operations executed*, where each operation costs a constant amount c_i .

Rather than adding up exact constants (which depend on the machine and implementation), we care about *how the total count grows as n grows* — which is where *asymptotic notation* comes in.

Cases of Analysis

Because the running time may depend not just on the *size* of the input but on its *specific values*, we analyze:

Worst case:

The input that causes the maximum number of operations.

Best case:

The input that causes the minimum number of operations.

Average case:

Expected running time over all inputs of size n .

Insertion Sort

INSERTION-SORT(A)

```
1  for j = 2 to n
2      key = A[j]
3      // Insert A[j] into the sorted sequence A[1..j-1]
4      i = j - 1
5      while i > 0 and A[i] > key
6          A[i+1] = A[i]
7          i = i - 1
8      A[i+1] = key
```

Insertion Sort

- The outer *for loop* iterates j from 2 to n . At the start of each iteration, $A[1..j-1]$ is already sorted.
- $\text{key} = A[j]$ saves the current element we want to insert.
- The inner *while loop* walks backwards through the sorted portion, shifting elements one position to the right as long as they are greater than key
- When the *while loop* ends, $A[i+1] = \text{key}$ drops key into its correct position.
- After this, $A[1..j]$ is sorted, setting up the next iteration correctly.

Proving Correctness

Loop Invariant: At the start of each iteration of the for loop (line 1), the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$ but in sorted order.

Two things are being claimed simultaneously:

- $A[1..j-1]$ is **sorted** (in nondecreasing order)
- $A[1..j-1]$ contains exactly the **original elements** that were in those positions (no elements have been lost or duplicated — they've just been rearranged)

Analyzing Running Time

The Cost Table

Line	Code	Cost	Times Executed
1	<code>for j = 2 to n</code>	c_1	n
2	<code>key = A[j]</code>	c_2	$n - 1$
4	<code>i = j - 1</code>	c_4	$n - 1$
5	<code>while i > 0 and A[i] > key</code>	c_5	$\sum_{j=2}^n t_j$
6	<code>A[i+1] = A[i]</code>	c_6	$\sum_{j=2}^n (t_j - 1)$
7	<code>i = i - 1</code>	c_7	$\sum_{j=2}^n (t_j - 1)$
8	<code>A[i+1] = key</code>	c_8	$n - 1$

Analyzing Running Time

Line 1 executes **n times**: the for loop header is evaluated once for each value $j \in \{2, 3, \dots, n\}$ plus one final time when $j = n+1$ to discover the loop should terminate. That's $(n-1) + 1 = n$ times.

t_j is defined as the number of times the while loop condition on line 5 is tested for a given value of j . This includes the final test that fails (causing the while loop to exit). So if the while body executes k times for a given j , then $t_j = k + 1$.

Line 6 and 7 (the while loop body) execute **$t_j - 1$** times for each j , because they run one fewer time than the condition is tested (the last test causes an exit without executing the body).

Analyzing Running Time

The Total Running Time Formula

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

This is exact. Now we need to evaluate $\sum_{j=2}^n t_j$, and this depends entirely on the input. This is where cases diverge.

Best Case Analysis

What input produces the best case?

An array that is *already sorted in nondecreasing order*.

Why? When A is already sorted, for every j , $A[j-1] \leq A[j] \leq A[j]$. So when we set $\text{key} = A[j]$ and check the while condition: $A[i] > \text{key}$ is immediately false (since $A[i] = A[j-1] \leq A[j] = \text{key}$). The while loop body never executes.

Therefore: $t_j = 1$ for each j (the condition is checked exactly once and fails).

$$\sum_{j=2}^n t_j = \sum_{j=2}^n 1 = n - 1$$

$$\sum_{j=2}^n (t_j - 1) = \sum_{j=2}^n 0 = 0$$

Best Case Analysis

Substituting:

$$T(n)$$

$$= c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_6 \cdot 0 + c_7 \cdot 0 + c_8(n - 1)$$

$$T(n) = c_1n + (c_2 + c_4 + c_5 + c_8)(n - 1)$$

$$T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

This is of the form:

$$T(n) = an + b$$

$$T_{Best}(n) = \Theta(n)$$

This is linear time. For every additional element you add to an already-sorted array, the cost grows by a fixed constant amount.

Worst Case Analysis

What input produces the worst case?

An array sorted in *strictly decreasing (reverse) order*.

Why? When the array is reverse sorted, every element $A[j]$ is smaller than all elements before it. So when we pick $\text{key} = A[j]$, the while loop must compare against every single element in $A[1..j-1]$ — none of them can be $\leq \text{key}$ — and shift all of them right.

Therefore: $t_j = j$ for each j .

The while condition is checked j times for each j : once for each of the $j-1$ elements that are larger than key , and one final time when $i = 0$ (when $i > 0$ fails).

$$\sum_{j=2}^n t_j = \sum_{j=2}^n j = (2 + 3 + 4 + \cdots + n)$$

Worst Case Analysis

Using the arithmetic series formula:

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n (t_j - 1) = \sum_{j=2}^n (j - 1) = (1 + 2 + 3 + \dots + (n - 1)) = \frac{n(n-1)}{2}$$

Substituting into the total cost formula:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1)$$

Expanding and grouping by powers of n:

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8)$$

Worst Case Analysis

$$T(n) = an^2 + bn + c$$
$$T_{\text{worst}}(n) = \Theta(n^2)$$

This is quadratic time. Doubling the input size roughly quadruples the running time.

Average Case Analysis

What input produces the average case?

We assume the input is a *random permutation* of n distinct numbers — each of the $n!$ permutations equally likely.

Computing the expected value of t_j :

For a given j , consider inserting $A[j]$ into the sorted subarray $A[1..j-1]$. Since the input is random, $A[j]$ is equally likely to be the smallest, 2nd smallest, ..., or j -th smallest among the j elements $\{A[1], \dots, A[j-1], A[j]\}$.

Average Case Analysis

If $A[j]$ is the largest among the j elements: the while loop never executes its body. $t_j = 1$.

If $A[j]$ is the 2nd largest: the while loop executes once. $t_j = 2$.

...

If $A[j]$ is the smallest: the while loop executes $j-1$ times. $t_j = j$.

Each case occurs with probability $1/j$. Therefore:

$$E[t_j] = \frac{1}{j}(1 + 2 + 3 + \dots + j) = \frac{1}{j} \cdot \frac{j(j+1)}{2} = \frac{j+1}{2}$$

Average Case Analysis

Summing:

$$\sum_{j=2}^n E[t_j] = \sum_{j=2}^n \frac{j+1}{2} = \frac{1}{2} \sum_{j=2}^n (j+1) = \frac{1}{2} (3 + 4 + \dots + (n+1)) = \frac{1}{2} \left(\frac{n(n+1)}{2} + n - 1 \right) \approx \frac{n^2}{4}$$

This is still *quadratic* roughly half of the worst case, but same asymptotic growth.

$$T_{avg}(n) = \Theta(n^2)$$

The average case does not rescue Insertion Sort. For random inputs, it is still quadratic.